Experiment-8

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_s
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv("/content/Bank Customer Churn Prediction.csv")
```

```
df.drop("customer_id", axis=1, inplace=True)
```

```
le = LabelEncoder()
for col in df.select_dtypes(include="object").columns:
    df[col] = le.fit_transform(df[col])
```

```
X = df.drop("churn", axis=1)
y = df["churn"]
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=
```

```
model = DecisionTreeClassifier(criterion="gini", max_depth=5, random_state=42)
model.fit(X_train, y_train)
```

```
DecisionTreeClassifier(max_depth=5, random_state=42)
```

```
y_pred = model.predict(X_test)
```

```
cm = confusion_matrix(y_test, y_pred)
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
```

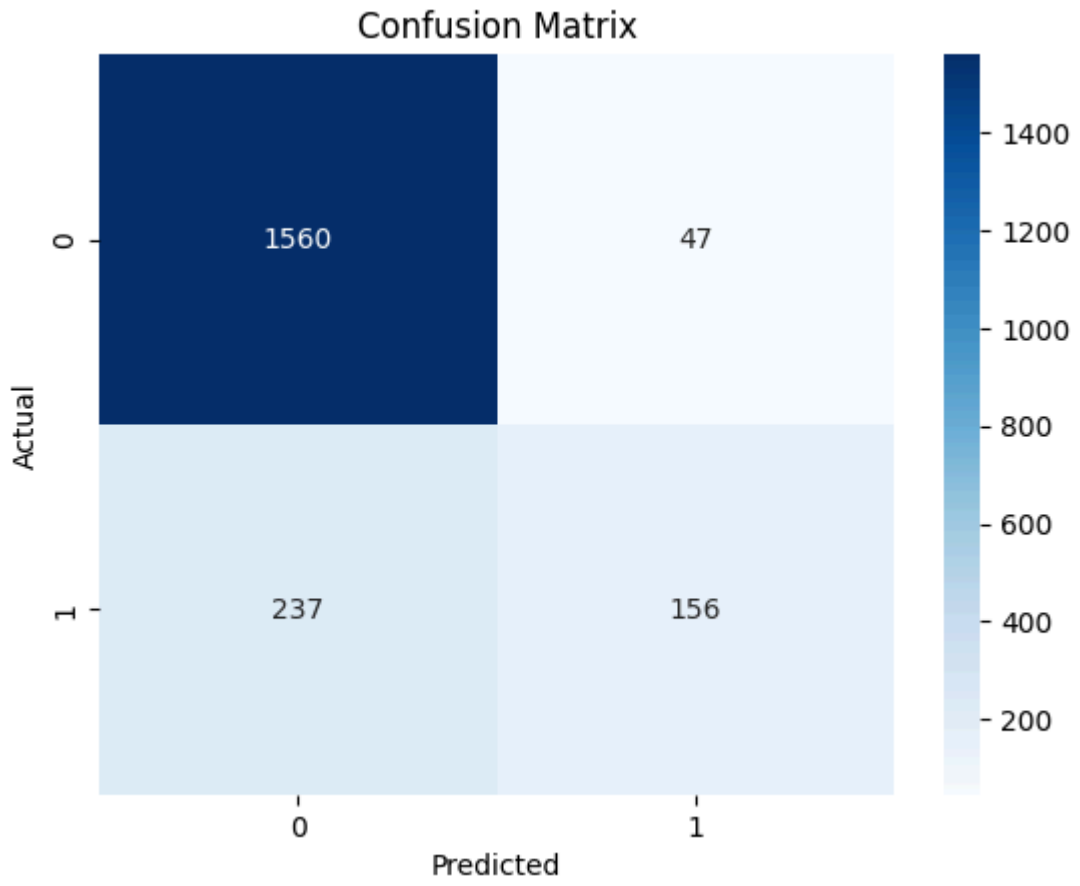
```
print("Confusion Matrix:\n", cm)
print("Accuracy:", acc)
print("Precision:", prec)
print("Recall:", rec)
print("F1 Score:", f1)
print(classification_report(y_test, y_pred))
```

```
Confusion Matrix:
 [[1560   47]
 [ 237  156]]
Accuracy: 0.858
Precision: 0.7684729064039408
Recall: 0.3969465648854962
F1 Score: 0.5234899328859061
              precision    recall  f1-score   support

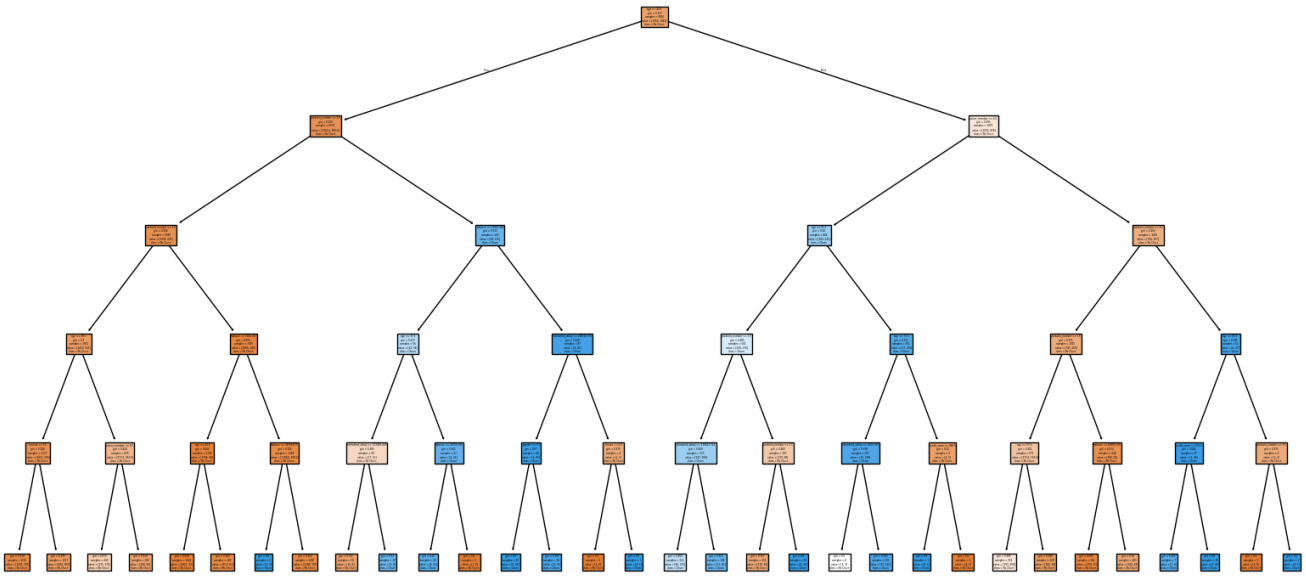
           0       0.87      0.97      0.92      1607
           1       0.77      0.40      0.52       393

    accuracy                           0.86      2000
   macro avg       0.82      0.68      0.72      2000
weighted avg       0.85      0.86      0.84      2000
```

```
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```



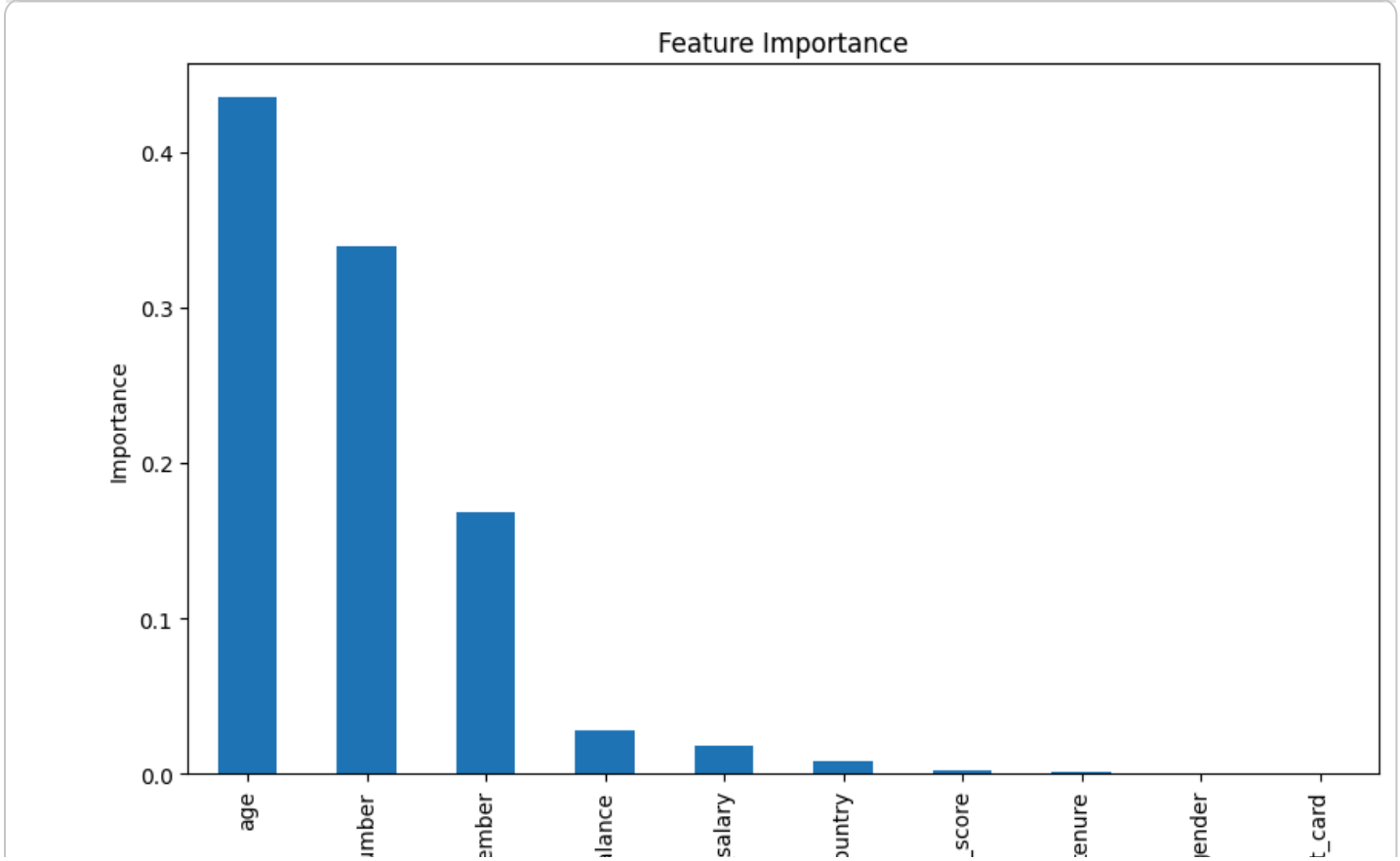

```
plt.figure(figsize=(20, 10))
plot_tree(model, feature_names=X.columns, class_names=["No Churn", "Churn"], filled=Tr
plt.show()
```



```
importances = pd.Series(model.feature_importances_, index=X.columns).sort_values(ascend
print(importances)
```

```
age                 0.434676
products_number     0.338766
active_member       0.168685
balance             0.027987
estimated_salary    0.017752
country             0.008125
credit_score        0.002021
tenure              0.001645
gender              0.000343
credit_card         0.000000
dtype: float64
```

```
plt.figure(figsize=(10, 6))
importances.plot(kind="bar")
plt.title("Feature Importance")
plt.ylabel("Importance")
plt.show()
```


Feature Importance
Importance
0.0
0.1
0.2
0.3
0.4
age
umber
amber
lance
salary
untry
score
enure
ender
t_card